

AUTOM-95 LAB: Terraform IAM Management

Series: AUTOM — Dynatrace Automation | **Reference:** 95 — Terraform IAM Management LAB | **Created:** May 2026 | **Last Updated:** 08/24/2026

Overview

This hands-on LAB walks the full lifecycle of managing Dynatrace **account-level IAM** with the `dynatrace-oss/dynatrace` Terraform provider: groups, policies, permission boundaries, and policy bindings. It assumes you have read **AUTOM-04** (Terraform Provider) and want to take IAM specifically to production.

Related lab, different question: this LAB covers the *provider lifecycle* — scaffold, resource types, DSL discovery, bulk export/import, CI/CD. For a worked, live-verified walkthrough that *provisions the recommended persona + team access model* (one shared `${bindParam:team}` policy template, per-team `dt.security_context` boundaries, environment-scoped bindings) with expected output from real runs, see **IAM-95** (Terraform flavor) and **IAM-96** (Python flavor) in the IAM series.

IAM is a separate concern from the tenant-level configuration covered in the main Terraform LAB (**AUTOM-98**):

Concern	Auth	API host	Provider scope
Tenant config (dashboards, alerting, settings)	Platform Token (<code>dt@s16</code>) and/or classic API Token (<code>dt@c01</code>)	<code><tenant>.live.dynatrace.com</code>	Most <code>dynatrace_*</code> resources
Account IAM (this LAB)	OAuth client credentials (<code>DT_CLIENT_ID</code> + <code>DT_CLIENT_SECRET</code> + <code>DT_ACCOUNT_ID</code>)	<code>api.dynatrace.com</code>	<code>dynatrace_iam_*</code> resources

The split is enforced by the provider, not by us — the `dynatrace_iam_group` resource (Dynatrace provider docs) explicitly requires "the environment variables `DT_CLIENT_ID` ,

`DT_CLIENT_SECRET` , `DT_ACCOUNT_ID` with an OAuth client." Platform Tokens and classic API tokens are rejected.

What you'll do

1. Create an OAuth client with the four minimal scopes
2. Stand up a Terraform scaffold for groups + policies + boundaries + bindings
3. Learn how to discover your account's permission DSL vocabulary (there is **no public catalog**)
4. Bulk-export an entire account's existing IAM as HCL via the provider's built-in `-export` utility
5. Import exported resources into Terraform state to bring them under management
6. Avoid the load-bearing gotchas (bindings re-assign all policies; `settings:objects:delete` doesn't exist; `terraform validate` doesn't parse the DSL)

Table of Contents

1. [OAuth Client Setup](#)
 2. [Provider Scaffold](#)
 3. [Groups](#)
 4. [Service Users](#)
 5. [Permission DSL Fundamentals](#)
 6. [Discovering Your Account's DSL Vocabulary](#)
 7. [Policies](#)
 8. [Verbs Are Per-Service \(`settings:objects:delete` Gotcha\)](#)
 9. [Boundaries](#)
 10. [Bindings \(`v2` — Re-Assigns All\)](#)
 11. [Bulk Export an Existing Account](#)
 12. [Importing Exported Resources](#)
 13. [Deprecated Arguments to Avoid](#)
 14. [CI/CD Integration](#)
 15. [Troubleshooting HTTP 400 — `TF\_LOG=DEBUG`](#)
 16. [Common Gotchas](#)
 17. [Production Considerations](#)
 18. [References](#)
-

Prerequisites

Requirement	Details
Terraform CLI	1.5 or later (verify with <code>terraform --version</code>)
Dynatrace SaaS account	Admin access required; Managed deployments are not supported by the IAM API
Account UUID	Visible in the URL on <code>account.dynatrace.com</code> (format: 8-4-4-12 hex, no <code>urn:dtaccount:</code> prefix)
Bash + <code>curl</code> + <code>jq</code>	For the optional DSL-discovery script in §6
AUTOM-04 read	This LAB assumes you've covered the Terraform Provider lecture (provider setup, combined auth, resource types)
AUTOM-98 LAB optional	The main Terraform hands-on. Useful but not required — this LAB is self-contained.

1. OAuth Client Setup

The IAM Account Management API requires an **OAuth2 bearer token**. Raw API tokens (`dt0c01.*`) and Platform Tokens (`dt0s16.*`) **will not work** — the API rejects them. The Terraform provider exchanges OAuth client credentials for short-lived bearer tokens automatically; you provide the client ID + secret + account UUID, the provider handles the rest.

Required OAuth client scopes

Scope	Needed for
<code>account-idm-read</code>	Read groups and users
<code>account-idm-write</code>	Create / delete groups
<code>iam-policies-management</code>	Create / update / delete policies, boundaries, bindings
<code>account-env-read</code>	Resolve environment references — required by the provider for <code>dynatrace_iam_policy</code> , <code>dynatrace_iam_policy_boundary</code> , and <code>dynatrace_iam_policy_bindings_v2</code> per the provider docs ("View environments (<code>account-env-read</code>)"), even when the policies are account-scoped

Verify scope names in the OAuth client creation UI — Dynatrace has occasionally renamed scopes between releases.

Creating the OAuth client

1. Open `https://account.dynatrace.com` in your browser
2. **Identity & Access Management** → **OAuth clients** → **Create client**
3. Description: e.g. `terraform-iam-management`
4. Grant the four scopes from the table above
5. Click **Create**
6. **Copy the Client ID and Client Secret immediately** — the secret is shown once and cannot be retrieved later. Store in a password manager / secret vault.
7. Note your **Account UUID** from the browser URL on `account.dynatrace.com`

Background reading: [OAuth clients \(DT docs\)](#).

Environment variables

```
export DT_CLIENT_ID="dt0s02.XXXXXXXX"
export DT_CLIENT_SECRET="dt0s02.XXXXXXXX.YYYYYYYY"
export DT_ACCOUNT_ID="abc12345-1234-1234-1234-abcdef012345"
```

`DT_ACCOUNT_ID` is your account UUID — the same value will appear in `terraform.tfvars` as `account_uuid` (Terraform variable). The provider reads it from the env var; resources also accept it as an HCL argument so bindings know which account scope to target.

To avoid shell history, source from a `chmod 600` env file:

```
cat &gt; ~/.dt-iam.env && 'EOF'
export DT_CLIENT_ID="..."
export DT_CLIENT_SECRET="..."
export DT_ACCOUNT_ID="..."
EOF
chmod 600 ~/.dt-iam.env
source ~/.dt-iam.env
```

2. Provider Scaffold

The scaffold uses one-resource-type-per-file — the same convention the `dynatrace-oss/dynatrace` provider's `-export` utility generates:

File	Purpose
versions.tf	Terraform + provider version constraints (pin the provider)
providers.tf	Provider block — auth via env vars only, no credentials in HCL
variables.tf	account_uuid , environment_id , management_zone_id
terraform.tfvars	Variable values (gitignored — never commit)
terraform.tfvars.example	Template for variable values (committed)
groups.tf	dynatrace_iam_group resources
policies.tf	dynatrace_iam_policy resources
boundaries.tf	dynatrace_iam_policy_boundary resources
bindings.tf	dynatrace_iam_policy_bindings_v2 resources

versions.tf

```

terraform {
  required_version = ">= 1.5"

  required_providers {
    dynatrace = {
      source = "dynatrace-oss/dynatrace"
      # Pin to the current line. Bump after verifying IAM resource args
      # against the new release notes – argument names have shifted in past
      # provider releases (see §13 Deprecated Arguments below).
      version = "~> 1.96"
    }
  }
}

```

providers.tf

```

# Authentication is read from environment variables – never put credentials
# in this file. See §1 for the required env vars and OAuth client scopes.
provider "dynatrace" {}

```

variables.tf

```
variable "account_uuid" {
  type      = string
  description = "Dynatrace account UUID – visible in the account.dynatrace.com URL.
  Format: 8-4-4-4-12 hex (e.g. abc12345-1234-1234-1234-abcdef012345). Do NOT include the
  urn:dtaccount: prefix."

  validation {
    condition     = can(regex("[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}$", var.account_uuid))
    error_message = "account_uuid must be a UUID in 8-4-4-4-12 hex format, without the
    urn:dtaccount: prefix."
  }
}

variable "environment_id" {
  type      = string
  description = "Dynatrace environment (tenant) ID – the subdomain portion of
  https://&lt;environment-id&gt;.live.dynatrace.com. Used to qualify management-zone-
  scoped permissions."
}

variable "management_zone_id" {
  type      = string
  description = "Management zone ID used by the production-only boundary example.
  Replace with a real MZ ID from your tenant before applying."
  default    = "REPLACE_WITH_REAL_MZ_ID"
}
```

terraform.tfvars.example

```
# Copy to terraform.tfvars and fill in real values.
# terraform.tfvars is gitignored – never commit real values.

account_uuid      = "abc12345-1234-1234-1234-abcdef012345"
environment_id    = "abc12345"
management_zone_id = "1234567890123456789"
```

Initialize

```
mkdir -p terraform/iam && cd terraform/iam
# (create the files above)
cp terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars with real values
terraform init
```

State will be local (`terraform.tfstate` in this directory) — **move to a remote backend before sharing this across operators**. AUTOM-09 §3 covers state-backend setup (S3 + DynamoDB / GCS / Azure Storage / HCP Terraform).

3. Groups (`dynatrace_iam_group`)

Groups are the unit of identity in Dynatrace IAM. Users (federated via SSO/IdP claims or manually added) belong to groups; permissions are granted to groups via policy bindings. **Names matter** when federating identity — match the group claim values your IdP emits (Azure AD, Okta, etc.) exactly.

Two example groups

```
# groups.tf

# Note: the deprecated `permissions` block is intentionally omitted.
# Use policy bindings (see §10) for permission assignment.

resource "dynatrace_iam_group" "platform_team" {
  name          = "platform-team"
  description = "Platform engineering – full admin access, scoped to production via boundary"
}

resource "dynatrace_iam_group" "dashboard_readers" {
  name          = "dashboard-readers"
  description = "Read-only access to monitoring data + dashboard editing"
}
```

`terraform apply` creates both groups in your account. They have no permissions yet — those come via bindings in §10.

4. Service Users (`dynatrace_iam_service_user`)

A **service user** is a non-human identity that holds IAM group memberships and is the identity *on whose behalf* automation runs. In production this is the identity your CI/CD pipelines act as, and the identity that owns the Platform Tokens minted for each job. The Terraform resource lets you provision the service user itself, attach it to one or more groups, and keep the whole identity model under version control alongside the groups and policies.

The 30-second mental model:

- **You manage with Terraform:** the service user resource + group memberships (this section).

- **You do NOT manage with Terraform:** the Platform Tokens themselves. The provider has no `dynatrace_platform_token` resource (verified against provider source 2026-05-22 — see AUTOM-04 §3). Mint Platform Tokens out-of-band via the DT UI (Account Management → Service Users) or via `POST /iam/v1/accounts/{accountId}/platform-tokens`, then deliver them to CI via a secret manager.

Resource block

Verbatim shape from the [provider docs](#):

```
# service-users.tf

resource "dynatrace_iam_service_user" "svc_terraform_ci" {
  name          = "svc-terraform-ci"
  description    = "CI/CD pipeline identity for Terraform applies"

  # Reference the groups created in §3 – Terraform handles the dependency order.
  groups = [
    dynatrace_iam_group.monitoring_admins.id,
  ]
}
```

Arguments

Argument	Required?	Notes
<code>name</code>	yes	Display name. Convention: prefix with <code>svc-</code> to keep service users distinguishable from human users in the UI listing.
<code>description</code>	optional	Strongly recommended — the description is what tells the next operator <i>why</i> this identity exists.
<code>groups</code>	optional	Set of group UUIDs. Reference via <code>dynatrace_iam_group.NAME.id</code> rather than hardcoded UUIDs — Terraform handles ordering and rebuild.
<code>email</code>	read-only	Dynatrace assigns a synthetic email. Surfaces in audit logs as the actor name.
<code>id</code>	read-only	The service user's UUID. Use this value as the <code>userId</code> body field when minting a Platform Token on behalf of the service user.

OAuth scopes — no additional cost over §1

Provisioning service users uses the same OAuth scopes §1 already lists for groups and policies — `account-idm-read` and `account-idm-write`. No new scope to add to your OAuth client for this resource.

Bulk export caveat — excluded by default

The `terraform-provider-dynatrace -export` utility *excludes* service users from the default export (per the provider docs). When bootstrapping Terraform management of an existing account, pass them explicitly:

```
./iam-export.sh dynatrace_iam_service_user
```

(Or modify the wrapper in §11 to always include them.)

Design recommendation — domain-scoped service users, not one mega-user

Resist the temptation to create a single `svc-terraform` service user that holds every IAM policy your automation needs. Two reasons:

1. **Least privilege.** A compromised credential on a mega-user can do anything your automation can do; domain-scoped users (e.g., `svc-tf-settings`, `svc-tf-iam`, `svc-tf-synth`) limit the blast radius to one functional area.
2. **Platform Token capacity headroom.** Dynatrace enforces a per-user-per-account cap on Platform Tokens. Domain separation multiplies your capacity ceiling proportionally — see [Platform tokens \(DT docs\)](#) for the current quota. A future AUTOM-07 section will cover the composed OAuth-client → ephemeral Platform Token pattern that exploits this separation.

A reasonable starting set for a Terraform-driven CI shop:

Service user	Group memberships (each binding via §10)	Typical CI use
<code>svc-tf-settings</code>	<code>monitoring-settings-writers</code>	Settings 2.0 management via <code>dynatrace_generic_setting</code>
<code>svc-tf-iam</code>	<code>iam-admins</code>	The bootstrap pipeline that applies this LAB itself
<code>svc-tf-synth</code>	<code>synthetic-writers</code>	Synthetic monitor lifecycle
<code>svc-tf-slo</code>	<code>slo-writers</code>	SLO and SRG definitions

Add fleet replicas (`svc-tf-settings-01`, `-02`, ...) only when one domain shows sustained concurrent-apply pressure against the Platform Token cap — not preemptively.

Sources: `dynatrace_iam_service_user` (Dynatrace provider docs), [Platform tokens \(DT docs\)](#). **Derived:** the domain-scoped service-user recommendation combines the provider's resource model with the Platform

Token per-user quota; neither source recommends this composition by name.

5. Permission DSL Fundamentals

Dynatrace IAM uses a small permission DSL inside `statement_query` strings. Format:

```
ALLOW <service>:<resource>:<action>[, ...] [WHERE  
<conditions>];
```

Multiple `ALLOW` / `DENY` statements can be combined in a single query, separated by semicolons:

```
ALLOW settings:objects:read, settings:schemas:read;  
DENY settings:objects:write WHERE settings:schemaId = "builtin:management-zones";
```

`WHERE` clauses support `=`, `IN (...)`, and boolean combinations on predicates such as `settings:schemaId`, `settings:schemaGroup`, `settings:scope`. They do **not** support arbitrary predicates like `document:type = "dashboard"` — narrow Gen 3 document scope via boundaries instead (see §9).

What the provider validates vs what Dynatrace validates

`terraform validate` and `terraform plan` confirm `statement_query` is a string with the right HCL shape. They do **not** parse the DSL inside the string. The first time Dynatrace sees the query content is at `terraform apply`, and unsupported tokens come back as **HTTP 400** with the response body stripped by the provider.

The practical implication: every new policy you write is one `apply` away from finding out whether your DSL strings parse. The next section is how to avoid that round trip.

6. Discovering Your Account's DSL Vocabulary

Dynatrace does not publish an exhaustive IAM permission catalog at a stable URL. The canonical reference is the set of policies *already in your account*. Dumping them and extracting the unique `service:resource:action` tokens gives you the authoritative vocabulary to copy from before writing any new policy.

The discovery script (`iam-list.sh`)

This script does the OAuth bearer-token exchange, fetches groups + policies + boundaries via the Account Management API, and prints a deduplicated list of every permission token used across all your existing policies.

```
#!/usr/bin/env bash
#
# iam-list.sh – Dump existing Dynatrace IAM groups, policies, and
# boundaries from an account. Print a deduplicated permission-token list
# (your account's canonical IAM DSL vocabulary).
#
# Required env vars: DT_CLIENT_ID, DT_CLIENT_SECRET, DT_ACCOUNT_ID
# OAuth client scopes: account-idm-read, iam-policies-management, account-env-read

set -euo pipefail

OUT="/tmp"
while [ $# -gt 0 ]; do
  case "$1" in
    -o|--output) OUT="$2"; shift 2 ;;
    -h|--help)   echo "Usage: $(basename "$0") [-o OUTPUT_DIR]"; exit 0 ;;
    *)           echo "Unknown arg: $1"; exit 2 ;;
  esac
done

: "${DT_CLIENT_ID:?DT_CLIENT_ID must be set}"
: "${DT_CLIENT_SECRET:?DT_CLIENT_SECRET must be set}"
: "${DT_ACCOUNT_ID:?DT_ACCOUNT_ID must be set}"

command -v jq &>/dev/null || { echo "ERROR: jq required"; exit 1; }
command -v curl &>/dev/null || { echo "ERROR: curl required"; exit 1; }

mkdir -p "$OUT"

TOKEN_URL="https://sso.dynatrace.com/sso/oauth2/token"
API_BASE="https://api.dynatrace.com"

echo "[1/5] Exchanging OAuth client credentials for a bearer token..."
TOKEN_RESPONSE=$(curl -sS -X POST "$TOKEN_URL" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=client_credentials" \
  --data-urlencode "client_id=$DT_CLIENT_ID" \
  --data-urlencode "client_secret=$DT_CLIENT_SECRET" \
  --data-urlencode "scope=account-idm-read iam-policies-management account-env-read" \
  --data-urlencode "resource=urn:dtaccount:$DT_ACCOUNT_ID")

TOKEN=$(echo "$TOKEN_RESPONSE" | jq -r '.access_token // empty')
[ -n "$TOKEN" ] || { echo "ERROR: token exchange failed:"; echo "$TOKEN_RESPONSE";
exit 1; }

H="Authorization: Bearer $TOKEN"

echo "[2/5] GET groups → $OUT/iam-groups.json"
curl -sS "$API_BASE/iam/v1/accounts/$DT_ACCOUNT_ID/groups" -H "$H" &> "$OUT/iam-
groups.json"

echo "[3/5] GET policy summaries → $OUT/iam-policies-list.json"
```

```

curl -sS "$API_BASE/iam/v1/repo/account/$DT_ACCOUNT_ID/policies" -H "$H" &gt;
"$OUT/iam-policies-list.json"

echo "[4/5] GET full policy details → $OUT/iam-policies-detail.json"
UUIDS=$(jq -r '.policies[]?.uuid // .items[]?.uuid // empty' "$OUT/iam-policies-
list.json")
echo "[]" &gt; "$OUT/iam-policies-detail.json"
for uuid in $UUIDS; do
    DETAIL=$(curl -sS "$API_BASE/iam/v1/repo/account/$DT_ACCOUNT_ID/policies/$uuid" -H
"$H")
    jq --argjson d "$DETAIL" '. += [$d]' "$OUT/iam-policies-detail.json" &gt; "$OUT/iam-
policies-detail.json.tmp"
    mv "$OUT/iam-policies-detail.json.tmp" "$OUT/iam-policies-detail.json"
done
echo "    Fetched $(jq 'length' "$OUT/iam-policies-detail.json") policy details"

echo "[5/5] GET boundaries → $OUT/iam-boundaries.json"
curl -sS "$API_BASE/iam/v1/repo/account/$DT_ACCOUNT_ID/boundaries" -H "$H" &gt;
"$OUT/iam-boundaries.json"

echo
echo "=== Files written ==="
ls -la "$OUT"/iam-*.json

echo
echo "=== Unique permission tokens used across all policies ==="
echo "(your account's canonical IAM DSL vocabulary)"
jq -r '.[].statementQuery // empty' "$OUT/iam-policies-detail.json" \
| grep -oE '[a-z][a-z0-9-]*:[a-z][a-z0-9-]*:[a-z][a-z0-9_-]*' \
| sort -u \
| sed 's/^/ /'

```

Save as `iam-list.sh`, `chmod +x`, and run:

```

./iam-list.sh                # writes JSON to /tmp/iam-*.json
./iam-list.sh -o ./out       # custom output directory

```

The final output is a deduplicated list of every `service:resource:action` token used in your account — for example:

```
account-idm:user:read
app-engine:apps:install
app-engine:apps:run
davis-copilot:queries:read
document:documents:admin
document:documents:delete
document:documents:read
document:documents:write
iam:policies:read
...
settings:objects:admin
settings:objects:read
settings:objects:write
settings:schemas:read
```

That list is your DSL Rosetta Stone. Copy verb conventions from it before authoring new policies.

Why this matters

A real-world account dump (133 policies, deduplicated to ~130 unique tokens) returned **zero occurrences** of `settings:objects:delete` — confirming that `:delete` is not a valid verb on `settings:objects` (the AWS-IAM-style extrapolation that produces an HTTP 400 at apply time). The script makes this kind of verification a 30-second routine.

7. Policies (`dynatrace_iam_policy`)

A policy bundles one or more `ALLOW` / `DENY` statements behind a name + description and scopes them to an account (or, in the deprecated pattern, an environment — see §13).

Three example policies

```
# policies.tf

resource "dynatrace_iam_policy" "monitoring_read_only" {
  name          = "monitoring-read-only"
  description   = "Read-only access to settings objects and schemas"
  account       = var.account_uuid

  statement_query = "ALLOW settings:objects:read, settings:schemas:read;"
}

# Documents in Dynatrace Gen 3 (dashboards, notebooks, segments) live in
# the `document:documents:*` namespace – not in Settings 2.0. This grants
# full edit (read/write/delete) on all documents in the account.
# For narrower scope (e.g. only dashboards owned by a specific group),
# use a boundary on document-level attributes.
resource "dynatrace_iam_policy" "dashboard_edit" {
  name          = "dashboard-edit"
  description   = "Edit documents (dashboards, notebooks, segments) – full CRUD"
  account       = var.account_uuid

  statement_query = "ALLOW document:documents:read, document:documents:write,
document:documents:delete;"
}

# `settings:objects:admin` is the canonical "full admin" verb for
# Settings 2.0 – it includes implicit delete (which is not exposed as a
# standalone token). Scope-limited via the production-only boundary in §10.
resource "dynatrace_iam_policy" "production_admin" {
  name          = "production-admin"
  description   = "Full settings admin via :admin – scope-limited via the production-
only boundary"
  account       = var.account_uuid

  statement_query = "ALLOW settings:objects:admin, settings:schemas:read;"
}
```

Apply and the policies exist in your account but are bound to **no groups** — bindings come in §10.

Best practice: Every policy's `statement_query` should use tokens that appeared in the `iam-list.sh` output from §6 against your account. If you need a verb that doesn't appear, you're either using a token name Dynatrace doesn't recognize, or you're working in an empty greenfield account (in which case write speculatively, apply, iterate until the API accepts).

8. Verbs Are Per-Service (`settings:objects:delete` Gotcha)

The DSL verb set is **not regular** across services. AWS-IAM-style extrapolation (`:read` / `:write` / `:delete` for every resource) does not work. The cleanest way to check is the `iam-list.sh` output from §6.

Verified verb sets from real account dumps:

Service:Resource	Valid verbs	Notes
<code>settings:objects</code>	<code>:read</code> , <code>:write</code> , <code>:admin</code>	No <code>:delete</code> . <code>:admin</code> is the umbrella verb covering full management including delete. Verified: 133-policy account dump returned 0 occurrences of <code>settings:objects:delete</code> .
<code>settings:schemas</code>	<code>:read</code>	Read-only — schemas are platform-defined, not user-managed.
<code>document:documents</code>	<code>:read</code> , <code>:write</code> , <code>:delete</code> , <code>:admin</code>	Gen 3 documents (dashboards, notebooks, segments) have <code>:delete</code> separately from <code>:admin</code> . Different verb set than <code>settings:objects</code> .
<code>app-engine:apps</code>	<code>:install</code> , <code>:run</code> , <code>:delete</code> , others	Verbs are domain-specific (<code>:install</code> / <code>:run</code> rather than <code>:read</code> / <code>:write</code>).
<code>iam:policies</code>	<code>:read</code> , <code>:write</code> , <code>:admin</code>	Yes, you can manage IAM with IAM.
<code>davis-copilot:queries</code>	<code>:read</code> , <code>:write</code>	etc.

The pattern: **verbs are per-service, not universal**. Always check against `iam-list.sh` output before writing.

The bug this prevents

```
# WRONG — produces HTTP 400 at apply with the body stripped by the provider
statement_query = "ALLOW settings:objects:read, settings:objects:write,
settings:objects:delete;"

# RIGHT — `:admin` is the umbrella verb that includes delete for settings:objects
statement_query = "ALLOW settings:objects:admin, settings:schemas:read;"
```

If apply fails on a policy with `HTTP 400` and no error body, re-run with `TF_LOG=DEBUG` (see §15).

9. Boundaries (`dynatrace_iam_policy_boundary`)

Boundaries are Dynatrace's analog to cloud-provider IAM conditions — they constrain *where* a permission applies, not *what* it does. The mental model transfers from AWS `Condition` blocks, Azure role-assignment scopes, or GCP IAM conditions, but the vocabulary is narrower: instead of IP ranges, request tags, or timestamps, Dynatrace boundaries operate on **environment**, **management zone**, **host tag**, and (for storage scopes) **bucket** and **security context**. If you're coming from AWS/Azure/GCP RBAC, don't look for IP-based or time-based conditions — they don't exist in this model. Plan boundary design around the dimensions Dynatrace actually exposes.

A boundary is a reusable `WHERE` clause that gets AND-ed onto a policy at binding time. It lets you write one policy granting (say) `settings:objects:admin` and then bind it with different boundaries for different groups — production admins get the full account, regional admins get one management zone, etc.

Common boundary patterns

```
# boundaries.tf

resource "dynatrace_iam_policy_boundary" "production_only" {
  name = "production-only"
  query = "environment:management-zone = \"${var.management_zone_id}\";"
}

# Other common patterns:
#   environment:management-zone startsWith "[Prod]"
#   environment:host-tag = "production"
#   environment:host-tag IN ("production", "prod-canary")
```

The `query` value is a WHERE clause without the `WHERE` keyword (the provider adds it during evaluation). String literals use escaped double quotes inside the HCL string.

Discovering boundary syntax beyond `environment:management-zone`

Boundary syntax is **less well documented than policy DSL syntax**. The discovery script in §6 also fetches boundaries (`/iam-boundaries.json`) — read existing boundaries in your account before writing new ones with novel predicates. The provider docs and the IAM docs do not exhaustively catalog the supported boundary predicates.

10. Bindings (`dynatrace_iam_policy_bindings_v2`)

A binding wires a group to a list of policies, optionally with boundaries applied per-policy. **One `dynatrace_iam_policy_bindings_v2` resource per (group, scope) pair.**

Two example bindings

```
# bindings.tf

# platform-team → production-admin, restricted to the production management
# zone by the production-only boundary.
resource "dynatrace_iam_policy_bindings_v2" "platform_team_admin" {
  group    = dynatrace_iam_group.platform_team.id
  account  = var.account_uuid

  policy {
    id          = dynatrace_iam_policy.production_admin.id
    boundaries = [dynatrace_iam_policy_boundary.production_only.id]
  }
}

# dashboard-readers → monitoring-read-only + dashboard-edit. No boundary
# (these are intentionally global to the account).
resource "dynatrace_iam_policy_bindings_v2" "dashboard_readers" {
  group    = dynatrace_iam_group.dashboard_readers.id
  account  = var.account_uuid

  policy {
    id = dynatrace_iam_policy.monitoring_read_only.id
  }

  policy {
    id = dynatrace_iam_policy.dashboard_edit.id
  }
}
```

Critical: `bindings_v2` re-assigns all policies on each apply

Per the [dynatrace_iam_policy_bindings_v2 resource docs \(Dynatrace provider docs\)](#) (same page in the provider repo (Dynatrace GitHub), which renders without JavaScript): *"This resource re-assigns all policies bound to a group, so every policy that should remain bound must be specified in the configuration; otherwise, it will be unbound."* The same page adds the consequence in its own words: *"During this process, there is a brief window where the group has no policies assigned, which may temporarily cause permission issues for users in that group."*

Three consequences:

1. If you remove a `policy {}` block from a `bindings_v2` resource, that policy is **unbound** from the group at the next apply.
2. If someone adds a policy via the UI, your next Terraform apply will **remove it**. This is a Dynatrace API behavior, not a Terraform one.
3. There is a **brief window during apply where the group has zero policies attached** — plan apply timing accordingly for production.

For groups whose policy set is partly Terraform-managed and partly UI-managed, this resource is a poor fit. Either bring all policy assignments under Terraform, or use the deprecated V1 bindings (`dynatrace_iam_policy_bindings`) which append rather than replace — though V1 is being phased out, see §13.

After apply

A successful apply against a fresh account leaves these 8 resources in state:

- 2 groups (`platform-team`, `dashboard-readers`)
- 1 boundary (`production-only`)
- 3 policies (`monitoring-read-only`, `dashboard-edit`, `production-admin`)
- 2 bindings (`platform_team_admin` with the production boundary, `dashboard_readers` without)

`terraform plan` against the now-current state should show **No changes** — the canonical sign that Terraform sees what the API sees.

11. Bulk Export an Existing Account

For accounts with existing IAM you want to bring under Terraform management, the `dynatrace-oss/dynatrace` provider ships with a built-in **export utility**. It's invoked by running the provider's compiled binary directly — not via `terraform <subcommand>`.

IAM is excluded from the default export — per the provider's own `-list-exclusions` output: *"Account management requires OAuth2 client and is specific to SaaS."* The string is defined in the provider source at `dynatrace/export/resource_descriptor.go` ([Dynatrace GitHub](#)); there is no docs page carrying it. To export IAM you must name the resources explicitly.

Step-by-step

Step 1. Create the OAuth client (already done in §1).

Step 2. Create a working directory + `versions.tf` + run `terraform init` — this downloads the provider binary that contains the export utility:

```
mkdir -p ~/dt-iam-export && cd ~/dt-iam-export

cat > versions.tf <<'EOF'
terraform {
  required_version = ">= 1.5"
  required_providers {
    dynatrace = {
      source = "dynatrace-oss/dynatrace"
      version = "> 1.96"
    }
  }
}
provider "dynatrace" {}
EOF

terraform init
```

Step 3. Confirm IAM is excluded by default (and see all exclusions):

```
.terraform/providers/registry.terraform.io/dynatrace-oss/dynatrace/*/terraform-
provider-dynatrace_v* \
  -export -list-exclusions
```

You'll see a section titled *"Account management requires OAuth2 client and is specific to SaaS"* listing the IAM resource types:

```
dynatrace_iam_user
dynatrace_iam_group
dynatrace_iam_permission
dynatrace_iam_policy
dynatrace_iam_policy_bindings
dynatrace_iam_policy_bindings_v2
dynatrace_iam_policy_boundary
```

Step 4. Set the environment variables:

```
export DT_CLIENT_ID="..."
export DT_CLIENT_SECRET="..."
export DT_ACCOUNT_ID="..."
# Required at provider startup even for IAM-only exports – the URL isn't
# actually used for IAM API calls. Use any valid tenant URL.
export DYNATRACE_ENV_URL="https://your-tenant.live.dynatrace.com"
```

Step 5. Run the export:

```
.terraform/providers/registry.terraform.io/dynatrace-oss/dynatrace/**/terraform-
provider-dynatrace_v* \
  -export -flat -id \
  dynatrace_iam_group \
  dynatrace_iam_policy \
  dynatrace_iam_policy_boundary \
  dynatrace_iam_policy_bindings_v2
```

Flag meanings:

Flag	Effect
<code>-export</code>	Switch the binary into export mode
<code>-flat</code>	Write all resources into one directory (no nested module structure) — easier to review
<code>-id</code>	Write each source resource's UUID as an HCL comment inside the file — crucial for <code>terraform import</code> in §12

For an account with ~100 groups and ~150 policies the export takes 30–90 seconds. Expected output:

```
Downloading "dynatrace_iam_policy_boundary" Count: 24
Downloading "dynatrace_iam_group" Count: 109
Downloading "dynatrace_iam_policy" Count: 145
Downloading "dynatrace_iam_policy_bindings_v2" Count: 122
Post-Processing Resources ...
Writing __datasources__.tf
Writing __variables__.tf
Writing main __providers__.tf
Finish Export ...
... finished after 47 seconds
```

Step 6. Review the output (defaults to `./.configuration/`, override with `DYNATRACE_TARGET_FOLDER`):

```
ls -la .configuration/
```

You'll find one `.tf` file per resource plus `__datasources__.tf` / `__variables__.tf` / `__providers__.tf`. Each resource block looks like:

```
# id = "abc12345-...-..."      &lt;-- the source UUID in a comment (because of -id)
resource "dynatrace_iam_group" "my_group" {
  name          = "my-group"
  description = "Some group"
}
```

The `id =` comment is your reference for `terraform import` in §12.

Step 7. Decide what to bring under management — see §12.

Optional wrapper script

For repeated use, this `iam-export.sh` wrapper handles binary discovery, env-var validation, and resource-list defaulting:

```
#!/usr/bin/env bash
#
# iam-export.sh – Bulk export Dynatrace IAM resources to HCL.
# Requires: terraform init in the same directory first.

set -euo pipefail

DEFAULT_RESOURCES=(
    dynatrace_iam_group
    dynatrace_iam_policy
    dynatrace_iam_policy_boundary
    dynatrace_iam_policy_bindings_v2
)

OUT_DIR="${PWD}/exported-iam"
EXTRA=()
while [ $# -gt 0 ]; do
    case "$1" in
        -o|--output) OUT_DIR="$2"; shift 2 ;;
        -h|--help)   echo "Usage: $(basename "$0") [-o OUTPUT_DIR]
[extra_resource_type...]" ; exit 0 ;;
        -*)          echo "Unknown flag: $1" ; exit 2 ;;
        *)           EXTRA+=("$1"); shift ;;
    esac
done

: "${DT_CLIENT_ID:?DT_CLIENT_ID must be set}"
: "${DT_CLIENT_SECRET:?DT_CLIENT_SECRET must be set}"
: "${DT_ACCOUNT_ID:?DT_ACCOUNT_ID must be set}"
: "${DYNATRACE_ENV_URL:?DYNATRACE_ENV_URL must be set (any valid tenant URL – required
at provider startup)}"

PROVIDER_DIR="${PWD}/.terraform/providers/registry.terraform.io/dynatrace-
oss/dynatrace"
[ -d "$PROVIDER_DIR" ] || { echo "ERROR: run 'terraform init' first"; exit 1; }

BINARY=$(find "$PROVIDER_DIR" -name "terraform-provider-dynatrace_v*" -type f | head
-1)
[ -n "$BINARY" ] || { echo "ERROR: provider binary not found; try 'terraform init -
upgrade'"; exit 1; }

RESOURCES=("${DEFAULT_RESOURCES[@]}")
if [ ${#EXTRA[@]} -gt 0 ]; then
    RESOURCES+=("${EXTRA[@]}")
fi

mkdir -p "$OUT_DIR"
export DYNATRACE_TARGET_FOLDER="$OUT_DIR"

echo "Provider binary: $BINARY"
echo "Output dir:      $OUT_DIR"
echo "Resources:       ${RESOURCES[*]}"
```

```

echo

(cd "$OUT_DIR" && "$BINARY" -export -flat -id "${RESOURCES[@]}")

TF_COUNT=$(find "$OUT_DIR" -type f -name "*.tf" | wc -l | tr -d ' ')
echo
echo "$TF_COUNT .tf files written under $OUT_DIR"
echo "Next: cd $OUT_DIR && ls (then $12 for import)"

```

Save as `iam-export.sh`, `chmod +x`, run from the directory where you ran `terraform init`. Add positional arguments to include extras: `./iam-export.sh dynatrace_iam_user dynatrace_iam_service_user`.

12. Importing Exported Resources Under Terraform Management

The exported files describe your account state **at the moment of export**. They do not populate Terraform state — every exported resource is foreign to your local state. To take over management:

1. **Pick which resources you want to manage.** Not all of them — ad-hoc test groups, deprecated bindings, and one-off service accounts are often not worth keeping in state.
2. **Copy the resource blocks you want into a working Terraform tree.** Add a real `providers.tf` and `versions.tf` if working in a new directory.
3. **Import each resource into state** with `terraform import <address> <id>`:
 - `<address>` is the resource block address — e.g. `dynatrace_iam_group.my_group`
 - `<id>` is the UUID from the `# id =` comment

```
bash terraform import dynatrace_iam_group.my_group abc12345-1234-1234-1234-abcdef012345
```

4. **Run `terraform plan`**. If the import worked, the plan should show **No changes** — Terraform sees the resource in state matching the resource in code.
5. **From now on, the resource is Terraform-managed.** Edits in HCL → `terraform apply`. Changes made in the Dynatrace UI will show up as drift in the next `terraform plan` — and a subsequent `apply` will revert them.

Scripting imports

There is no `terraform import-all` command. For 100+ resources, write a shell loop that reads each file's `# id =` comment:

```

for tf_file in *.tf; do
    resource_block=$(grep -E '^resource "[^"]+" "[^"]+"' "$tf_file" | head -1 | awk -F'"' '{print $2"."$4}')
    source_id=$(grep -E '^# id =' "$tf_file" | head -1 | sed -E 's/\.*= *"([^\"]+)"/\1/')
    if [ -n "$resource_block" ] && [ -n "$source_id" ]; then
        terraform import "$resource_block" "$source_id"
    fi
done

```

Test on a single file first. **Some resources have compound IDs** (UUID + account UUID joined by `#-#`) — those need the full compound string from the source `id`, not just the UUID.

Alternative: `-import-state` instead of `-flat -id`

The provider supports `-import-state` instead of `-flat -id`, which auto-runs `terraform init` and imports the exported resources into state in one step. It's opinionated about target module structure — read the provider's export docs first if you take this path. For a learning LAB, the explicit `terraform import` flow above is more instructive.

13. Deprecated Arguments to Avoid

The provider has flagged these deprecations since v1.96. Do **not** re-add them:

Deprecated	Replaced by	Notes
<pre> dynatrace_iam_group.permissions {} block </pre>	Policies + <pre> dynatrace_iam_policy_bindings_v2 </pre>	The block let you grant permissions inline on a group. Use bindings instead — they decouple groups from policies and let multiple groups reuse the same policy.

Deprecated	Replaced by	Notes
<code>dynatrace_iam_policy.environment</code> argument	<code>account = var.account_uuid</code>	The <code>environment</code> argument let you scope a policy to one tenant. Account-scoped policies are the current model — they work across all tenants in the account.
<code>dynatrace_iam_policy_bindings</code> (V1, no suffix)	<code>dynatrace_iam_policy_bindings_v2</code>	V1 appended policies; V2 re-assigns the full list. V2 is the current recommendation.

Why argument names shift between releases

IAM provider resources have had argument-name changes in past releases. Before bumping the provider version pin in `versions.tf`:

1. Run `terraform init -upgrade` and note the resolved version
2. Open the [dynatrace-oss/dynatrace provider releases \(GitHub\)](#) and read the notes for every version between the pinned one and the new one
3. Check the registry docs for each IAM resource you use — argument names, deprecations, new required fields
4. Re-run `terraform plan` and confirm the diff matches your expectation

A planned upgrade is the only kind that doesn't blow up at apply time.

14. CI/CD Integration (Brief)

Running IAM Terraform from a CI/CD pipeline follows the same pattern as the tenant-level Terraform LAB (**AUTOM-96**) — with these IAM-specific adjustments:

Concern	Tenant-level (AUTOM-96)	IAM (this LAB)
Auth source	OIDC → Vault → Platform Token (<code>dt0s16</code>)	OIDC → Vault → OAuth client credentials (<code>DT_CLIENT_ID</code> + <code>DT_CLIENT_SECRET</code>)

Concern	Tenant-level (AUTOM-96)	IAM (this LAB)
Required env vars	<code>DYNATRACE_ENV_URL</code> , <code>DYNATRACE_PLATFORM_TOKEN</code> (or <code>DYNATRACE_API_TOKEN</code>), <code>DYNATRACE_HTTP_OAUTH_PREFERENCE=true</code>	<code>DT_CLIENT_ID</code> , <code>DT_CLIENT_SECRET</code> , <code>DT_ACCOUNT_ID</code> (no <code>DYNATRACE_HTTP_OAUTH_PREFERENCE</code> needed — IAM is OAuth-only by construction)
State backend	Per-env state file (e.g. <code>s3://bucket/<env>/terraform.tfstate</code>)	Single account-level state file (<code>s3://bucket/iam/terraform.tfstate</code>) — IAM is account-wide, not per-env
Plan-and-apply gate	PR comment with <code>terraform plan</code> output; protected environment for apply	Same pattern, but apply gate should require second human approver — IAM changes affect every tenant in the account
Drift detection	<code>terraform plan -detailed-exitcode</code> on a schedule	Same — but more important here because UI changes to IAM trigger immediate revert on next apply (see §10 bindings caveat)

For the full CI/CD pattern with worked YAML, see **AUTOM-96 LAB: GitHub Actions CI/CD for Dynatrace Terraform**. The auth-injection mechanics are the same — substitute OAuth client credentials for the Platform Token, drop the `DYNATRACE_HTTP_OAUTH_PREFERENCE` env var, and you have a working IAM pipeline.

Single SA writer rule applies

Per **AUTOM-07 § Operational Model**, your account should have a single Service Account (here: the OAuth client) that is the **only writer** of IAM via Terraform. Humans read-only via the UI; all changes flow through PRs → CI → apply by the SA. This makes the audit trail clean (every IAM change is a Git commit) and lets you turn off all human IAM writes at the IAM layer itself.

15. Troubleshooting HTTP 400 — `TF_LOG=DEBUG`

When `terraform apply` fails on an IAM resource with `HTTP 400` and **no error body**, the provider has stripped Dynatrace's actual error message. To see what Dynatrace actually said, re-run with debug logging:

```
TF_LOG=DEBUG terraform apply &&1 | tee apply.log
```

Then search the log for `HTTP/2.0 400` (or `HTTP/1.1 400` for older Terraform versions):

```
grep -A 20 'HTTP/2.0 400' apply.log | less
```

The 20-line context window after the status line usually contains the JSON response body.
Common patterns:

Error body fragment	Likely cause	Fix
"Invalid permission" + a token name	Token not in your account's DSL vocabulary	Run <code>iam-list.sh</code> from §6; copy a verified token
"Boundary not found"	Boundary ID in a binding doesn't match a real boundary	Check the binding's <code>boundaries = [...]</code> IDs against <code>terraform state list</code>
"Group not found"	Group ID in a binding doesn't match a real group	Same as above for groups
"Policy parse error"	DSL syntax error in <code>statement_query</code> (missing semicolon, malformed <code>WHERE</code>)	Re-check the policy against the format in §5

If the `apply.log` doesn't include the response body even with `TF_LOG=DEBUG` , also set:

```
export DYNATRACE_DEBUG=true
export DYNATRACE_HTTP_RESPONSE=true
export DYNATRACE_LOG_HTTP=terraform-provider-dynatrace.http.log
TF_LOG=DEBUG terraform apply &&1 | tee apply.log
```

`DYNATRACE_HTTP_RESPONSE=true` tells the provider not to strip response bodies on error.
`DYNATRACE_LOG_HTTP` writes a separate, complete HTTP transcript to a file alongside the run.

16. Common Gotchas

Consolidated from §§ 4 through 14. Save this section for quick reference.

#	Gotcha	Why	Mitigation
1	Platform Tokens and classic API tokens don't work for IAM	IAM API requires OAuth2 bearer token; the provider enforces this per the resource docs	Use OAuth client credentials (<code>DT_CLIENT_ID</code> + <code>DT_CLIENT_SECRET</code> + <code>DT_ACCOUNT_ID</code>) only
2	<code>terraform validate</code> accepts invalid DSL	The provider treats <code>statement_query</code> as an opaque string; Dynatrace parses at apply	Run <code>iam-list.sh</code> (§6) to get the verified vocabulary before writing

#	Gotcha	Why	Mitigation
3	<code>settings:objects:delete</code> doesn't exist	DSL verb sets are per-service, not regular (verified: 133-policy dump returned 0 occurrences)	Use <code>settings:objects:admin</code> as the umbrella verb for full management
4	<code>document:type = "dashboard"</code> WHERE predicate isn't supported	DSL WHERE clauses only support a fixed set of predicates (<code>settings:*</code> , <code>environment:*</code>)	Narrow Gen 3 document scope via boundaries on document attributes
5	<code>bindings_v2</code> re-assigns all policies on each apply	Documented Dynatrace API behavior, not a Terraform bug	Every policy that should remain bound must be in the <code>bindings_v2</code> config
6	UI-added policies get wiped by next Terraform apply	Same as #5	Make Terraform the single writer of IAM; UI is read-only
7	Brief window during apply where group has zero policies	Re-assignment isn't atomic	Schedule applies during low-impact windows for production
8	HTTP 400 with no error body	Provider strips Dynatrace's error responses by default	<code>TF_LOG=DEBUG + DYNATRACE_HTTP_RESPONSE=true</code> (§15)
9	Export utility excludes IAM by default	<i>"Account management requires OAuth2 client and is specific to SaaS"</i>	Name the IAM resource types explicitly: <code>-export -flat -id dynatrace_iam_group dynatrace_iam_policy dynatrace_iam_policy_boundary dynatrace_iam_policy_bindings_v2</code>
10	<code>DYNATRACE_ENV_URL</code> required at startup even for IAM-only export	Provider initialization quirk; URL isn't used for IAM API calls	Set to any valid tenant URL in the account
11	Generated HCL is point-in-time	Re-running <code>-export</code> produces fresh files, not incremental diffs	Treat export as a discovery tool, not a sync mechanism

#	Gotcha	Why	Mitigation
12	Some resources have compound IDs (<code>UUID#-#account_uuid</code>)	Resource-specific Dynatrace convention	Use the full compound string from the source <code>id</code> when running <code>terraform import</code> , not just the UUID
13	Argument names shift between provider releases	IAM resources have had renames in past major releases	Pin a version in <code>versions.tf</code> ; read release notes before bumping (§13)
14	Local state is a single point of compromise	If state leaks, your IAM is exposed (and account UUID + group IDs are visible)	Move to remote backend with at-rest encryption + IAM restrictions (AUTOM-09 §3)
15	Greenfield accounts have no DSL vocabulary to copy from	<code>iam-list.sh</code> returns nothing on an empty account	Write the first policy speculatively (<code>ALLOW settings:objects:read, settings:schemas:read</code> ; is the safest universal starting point) and iterate

17. Production Considerations

Before relying on this scaffold in production:

Topic	Action	Cross-reference
State backend	Move from local state to remote (S3 + DynamoDB / GCS / Azure Storage / HCP Terraform). Single account-level state file.	AUTOM-09 §3
State encryption	At-rest encryption on the state bucket; IAM restriction so only the CI service principal can read	AUTOM-09 §3
CI/CD	Plan-on-PR, apply-on-merge, environment-protected approval	AUTOM-96 LAB
Single writer rule	OAuth client is the <i>only</i> writer of IAM via Terraform. UI is read-only for humans.	AUTOM-07 § Single SA Writer
OAuth client rotation	OAuth client secret on a schedule (90 days recommended). Update the secret in your secret manager; no Terraform change needed.	—
Drift detection	<code>terraform plan -detailed-exitcode</code> on a schedule; alert when exit code is 2 (drift detected)	AUTOM-09 §11

Topic	Action	Cross-reference
Backup	Run <code>iam-export.sh</code> on a schedule into a backup repo — your account's state as committed HCL is a low-cost insurance policy	§11
Account UUID + group IDs in code	These aren't secrets but they're identifying — treat the repo as "internal" rather than open-source	—
Boundary catalog discoverability	Boundary syntax beyond <code>environment:management-zone</code> isn't well documented — keep a per-account boundary cheat-sheet derived from <code>iam-list.sh</code> output	§9

18. References

Provider documentation

- [dynatrace-oss/dynatrace](#) provider (Terraform Registry)
- [dynatrace_iam_group](#) resource (Dynatrace provider docs)
- [dynatrace_iam_policy](#) resource (Dynatrace provider docs)
- [dynatrace_iam_policy_boundary](#) resource (Dynatrace provider docs)
- [dynatrace_iam_policy_bindings_v2](#) resource (Dynatrace provider docs)
- [Provider GitHub repository](#) (Dynatrace GitHub) — `dynatrace/export/initialize.go` is the source-of-truth for export flags
- [Provider releases](#) (Dynatrace GitHub) — read release notes before bumping pinned version

Dynatrace IAM documentation

- [Manage user permissions: policies](#) (DT docs)
- [IAM policy statements reference](#) (DT docs)
- [OAuth clients](#) (DT docs)
- [Terraform CLI commands — Export utility](#) (DT docs)

Adjacent AUTOM notebooks

- **AUTOM-04** — Terraform Provider lecture (provider config, combined auth, resource types overview)
- **AUTOM-07** — CI/CD Integration (Single SA Writer rule; cross-platform CI/CD recipes)
- **AUTOM-09** — Terraform GitOps Setup Recipe (state backends, modules, lifecycle protections)
- **AUTOM-96 LAB** — GitHub Actions CI/CD for Dynatrace Terraform (full plan-and-apply pipeline)

- **AUTOM-98 LAB** — Terraform for Dynatrace (general hands-on; this LAB-95 zooms in on IAM)

Continue to AUTOM-09: Terraform GitOps Setup Recipe for state-backend setup and lifecycle protections that apply to IAM the same way they apply to tenant config; or AUTOM-96 LAB for the CI/CD pipeline pattern.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.